التوزيع والتجميع في سحاب الجميع

Lecture 5 — Distributed Systems and Cloud-Native Software

Q/ Why distribute ?

1. **Scale**: data and traffic exceed what one server can store or process.
2. **Availability**: if one machine dies, another continues.
3. **Locality**: place data near users and near the computation to cut latency.

> The price: machines fail independently, messages are delayed or lost, and clocks disagree.

Q/ What can go wrong in a distributed system ?

Failure	Meaning	Example
Crash	A node stops and may restart	Power loss, kernel panic
Message loss / delay	Arrives late, twice, or never	Congested link
Network partition	Cluster splits into groups that can't talk	Switch or inter-site link fails
Clock skew	Nodes disagree about time	Drifting clocks
Byzantine	A node lies or behaves arbitrarily	Compromised or buggy node

> Raft and Paxos tolerate crashes and lost messages, **not Byzantine nodes** — authenticate membership.

Consensus

> **Consensus** : a group of nodes agrees on a **single value**, even when some nodes fail.

Q/ Where is consensus needed ?

1. Electing **one leader**.
2. A **replicated log** of operations in the same order.
3. Distributed locks and configuration.
4. Committing a transaction across nodes.

Q/ What are the required properties of consensus ?

1. **Agreement**: no two nodes decide differently.
2. **Validity**: the decided value was proposed by someone.
3. **Termination**: every working node eventually decides.

> **FLP result** : no algorithm can guarantee agreement in a fully asynchronous network with even one crash — real protocols use **timeouts**.

> **Quorum (majority)** : any two majorities share at least one node, so two groups can never decide differently.

Formula: **N = 2f + 1** nodes survive **f** crashes.

Nodes (N)	Majority	Failures tolerated
3	2	1
4	3	1
5	3	2
7	4	3

Use an **odd** number: 4 nodes tolerate only 1 failure, same as 3, and cost more. A partition leaves at most **one** side with a majority.

Paxos and Raft

> **Paxos (Lamport)** : the classic consensus algorithm — proven correct but famously hard to understand.

Q/ What are the Paxos roles ?

1. **Proposer**: suggests a value and drives the protocol.
2. **Acceptor**: votes; a value is chosen when a **majority** of acceptors accept it.
3. **Learner**: finds out which value was chosen and applies it.

Q/ What are the two Paxos phases ?

1. **Prepare**: proposer sends Prepare(n); acceptors reply **Promise(n)** — ignore lower numbers, report any value already accepted.

2. **Accept**: with a majority of promises, proposer sends Accept(n, v); a majority of accepts → v is chosen.

Multi-Paxos (stable leader) is used in Google **Chubby** and **Spanner**, and **Ceph** monitors.

> **Raft** : consensus designed to be **understandable**, with **one strong leader** that replicates every change to followers.

Q/ What are the three parts of Raft ?

1. **Leader election**. 2. **Log replication**. 3. **Safety**.

> **Term** : a logical clock — each election starts a new term.

> **Heartbeat** : an empty AppendEntries the leader sends regularly to stay in charge.

Q/ What are Raft's two RPCs ?

- **RequestVote** (elections) and **AppendEntries** (replication + heartbeat).

Q/ What are the Raft node states and transitions ?

- **Follower** → (election timeout) → **Candidate** → (majority of votes) → **Leader**; a leader/candidate that sees a **higher term** steps down to follower.

Q/ How does leader election work ?

1. **Timeout**: the leader goes silent; followers wait a **random** timeout (~150–300 ms).
2. **Candidate**: the first to time out increments the term, votes for itself, asks for votes.
3. **Vote**: each node votes at most **once per term**.
4. **Leader**: with a majority (e.g. 3 of 5) it becomes leader and sends heartbeats.

Split vote → nobody wins → new election; random timeouts make another tie unlikely.

Q/ How does log replication work ?

1. Client sends a command to the leader.
2. Leader appends it to its own log.
3. Leader sends **AppendEntries** to followers.
4. When a **majority** has stored it → **commit**.
5. Apply to the state machine and reply to the client.

Raft under failure	What happens	Why safe
Leader crashes	Followers time out, elect a new leader in a higher term	Committed entries are on a majority
Follower crashes	Leader continues while a majority lives; follower catches up later	No decision needs every node
Partition 3 | 2	3-side keeps a leader; 2-side can't commit	Only one side has a majority → **no split brain**
Old leader returns	Sees a higher term, steps down	Stale leaders can't commit

> **Election restriction** : a candidate whose log is behind **cannot win**, protecting committed data.

System	Protocol	Keeps in agreement
etcd	Raft	All Kubernetes cluster state, including Secrets
Consul	Raft	Service discovery and configuration
ZooKeeper	ZAB (Paxos-like)	Coordination for Hadoop, Kafka, HBase
Ceph	Paxos	Cluster map held by monitors
Spanner, CockroachDB	Paxos, Raft	Replicated data in global SQL

Use a proven system (e.g. etcd) — **don't write your own consensus**.

Consistency and CAP

Level	Guarantee	Example	Cost
Strong (linearizable)	Every read returns the latest write	Bank balance, leader lock	Highest latency; may refuse requests in a partition
Sequential	Same order for everyone, not necessarily real time	Replicated log	Slightly cheaper

Level	Guarantee	Example	Cost
Causal	If A caused B, everyone sees A before B	Comment after its post	Moderate
Eventual	Replicas converge when writes stop; reads may be stale	DNS, like counters, cart	Fastest, most available

Security: with eventual consistency a **revoked permission** can still be honored by a lagging replica — use **strong reads for authorization**.

> **CAP theorem** : in a network **Partition**, you must choose **Consistency** (reject some requests) or **Availability** (answer, maybe stale).

Choice	During a partition	Examples
CP	Minority side refuses requests	etcd, ZooKeeper, HBase
AP	Every side answers; reconcile later	Cassandra, DynamoDB, CouchDB

> **PACELC** : without a partition, the trade-off is **latency vs consistency**.

Cloud-native software

Aspect	Conventional	Cloud-native
Architecture	One large **monolith**	Small independent **microservices**
Runtime	Installed on a specific server/VM	Containers managed by an orchestrator (Kubernetes)
Scaling	Scale **up** (bigger server)	Scale **out** (more small instances, automatic)
Failure	Avoided; assumes reliable hardware	**Expected**; designed to self-heal
State	Stored on the local server	Services **stateless**; state in external stores
Releases	Rare, large, manual	Frequent, small, automated (**CI/CD**)

Cloud-native ≠ "running in the cloud". It means **designed to exploit** the cloud: elastic, resilient, automated.

MapReduce and Hadoop

> **MapReduce** : a programming model — split huge data, process pieces in parallel (**map**), group by key (**shuffle and sort**), then combine (**reduce**).

map: (k1, v1) → list(k2, v2) · reduce: (k2, list(v2)) → list(v3)

Q/ Word count example — lines "cloud is big", "cloud is shared", "big data is big" ?

1. **Map** emits (word, 1) for every word.
2. **Shuffle** groups: big:[1,1,1] cloud:[1,1] data:[1] is:[1,1,1] shared:[1].
3. **Reduce** sums: big 3, cloud 2, data 1, is 3, shared 1 (total 10).

Q/ What are two key ideas of MapReduce ?

1. **Bring computation to the data**: map tasks run on the machines that already store the block.
2. **Failures are handled**: a failed or slow task is simply re-run elsewhere.

> **Amdahl's law** : S(N) = 1 / ((1 − p) + p / N), p = parallel fraction, N = machines.

Example: p = 0.95, N = 100 → S ≈ **16.8**, not 100. Job time ≈ T_map/N + T_shuffle + T_reduce/R — the **shuffle and serial part don't shrink**.

> **Hadoop** : three layers — **HDFS** (distributed storage), **YARN** (resource scheduling), **MapReduce** (processing engine).

> **HDFS** : files cut into **128 MB blocks**, each stored on **3 DataNodes**; the **NameNode** holds metadata (file → blocks → DataNodes); clients read directly from DataNodes.

Q/ How is the NameNode single point of failure solved ?

- **Active + standby** NameNode, edits kept in a quorum of **JournalNodes**, failover coordinated by **ZooKeeper** (consensus again).

Languages: Java natively; **Hadoop Streaming** lets any stdin/stdout program (Python, Perl) be a mapper/reducer; **Hive** (SQL-like) and **Pig** (scripting) add layers.

Security lens

Area	Typical risk	Good practice
Consensus cluster (etcd, ZooKeeper)	Unauthenticated members; whoever controls a majority controls the state (incl. Kubernetes Secrets)	Mutual TLS, client auth, encryption at rest, isolation
Consistency & CAP	Stale reads honour revoked permissions; split brain	Strong reads for authz, short token lifetimes, fencing tokens
Hadoop & HDFS	Open web UIs/RPC ports, weak default auth	**Kerberos**, ACLs, encryption zones, audit logs
Cloud-native apps	Vulnerable images, secrets in code, over-privileged accounts, supply chain	Image scanning & signing, secret managers, least privilege
MapReduce jobs	Untrusted code; poisoned input	Tenant isolation, quotas, input validation, sandboxing

Q/ Mini case — exam results replicated across three sites: grades vs "last login" ?

1. Grades → **strong / CP** (must never be stale or conflicting).
2. Last login → **eventual / AP** is acceptable.
3. Raft with **3 nodes, one per site** (or 5 across sites): if one site is cut off, the other two keep a majority and continue; the isolated site can't commit.
4. Failed logins with MapReduce: **map** emits (studentId, 1) for each failed-login line; **reduce** sums per student and outputs those above a threshold.

Summary of Cloud Computing L5: Distributed Systems & Cloud-Native. Built only from the lecture file; check the original slides for anything you're unsure about.